Mod 4

Convolutional Neural Networks

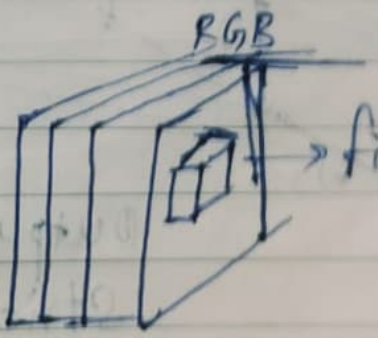
Convolution is a mathematical way of combining two signals to form a third signal, it is one of the most important techniques in signal processing.

In case of 2D data (grayscale images) the convolution operation between a filter $W^{k \times k}$ and an image $X^{N_1 \times N_2}$ can be expressed as:

$$Y(i,j) = \sum_{u=-k}^{k} \sum_{v=-k}^{k} W(u,v) X(i-u, j-u).$$

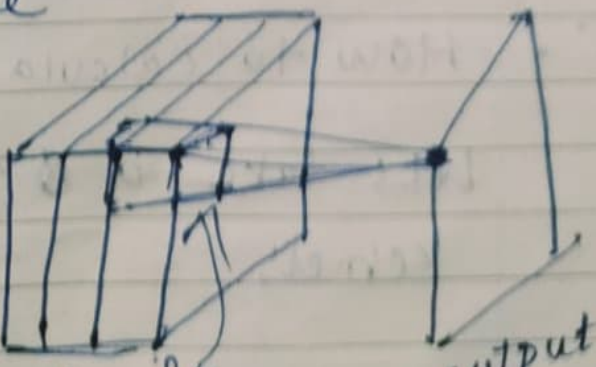
What would a 3D filter look like?

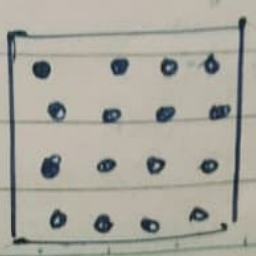
- It will be in 3D too and will refer to it as a volume.
- We keep sliding this volume over 3D input and perform convolution.



We always assume that the filter extends till the depth of the image

In effect output is 2D (width & height).

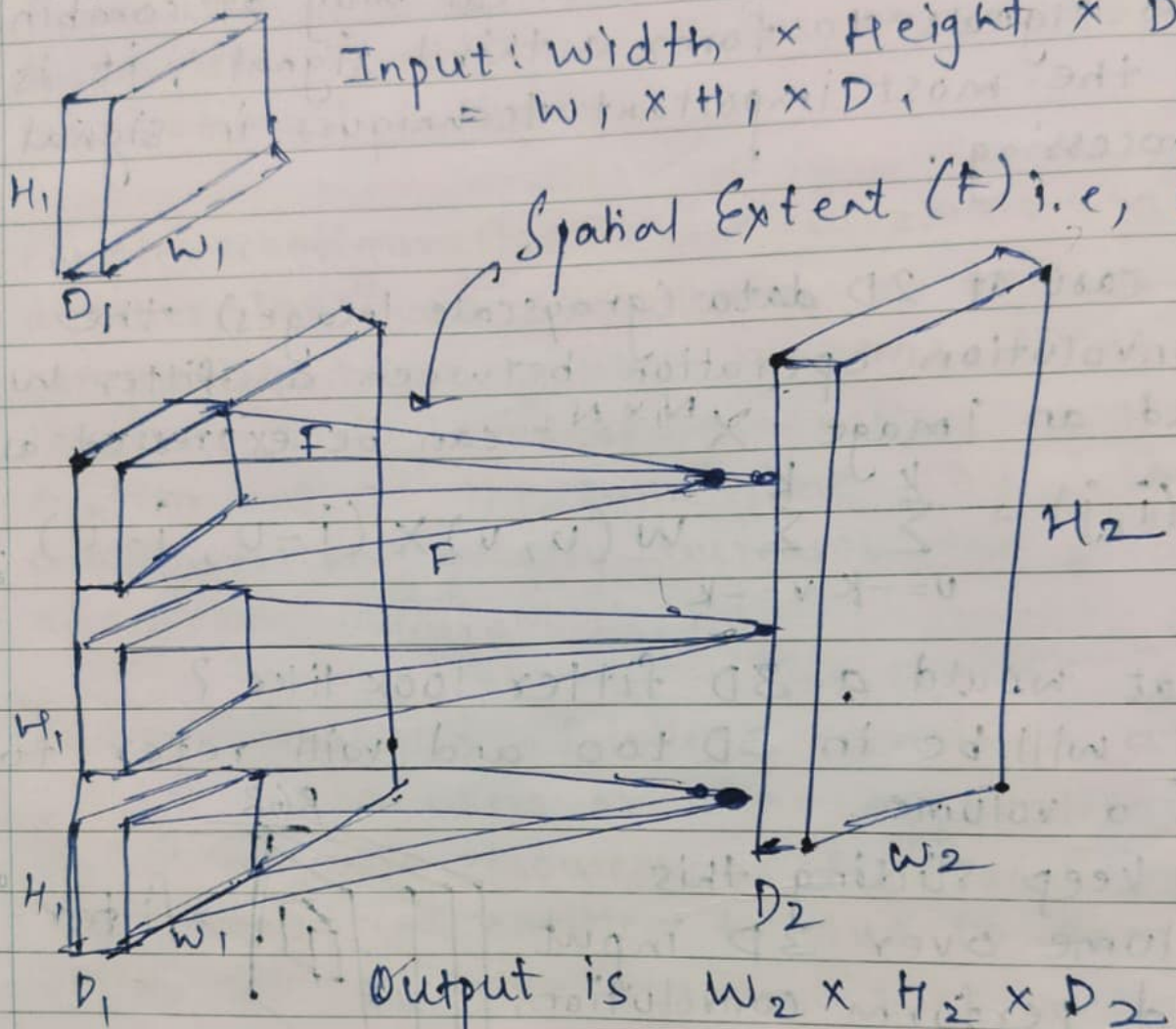




to get output

we can apply multiple ~~features~~ filters at multip to get multiple feature maps.

Hyperparameters of CNN



Output is $W_2 \times H_2 \times D_2$

Stride (S)

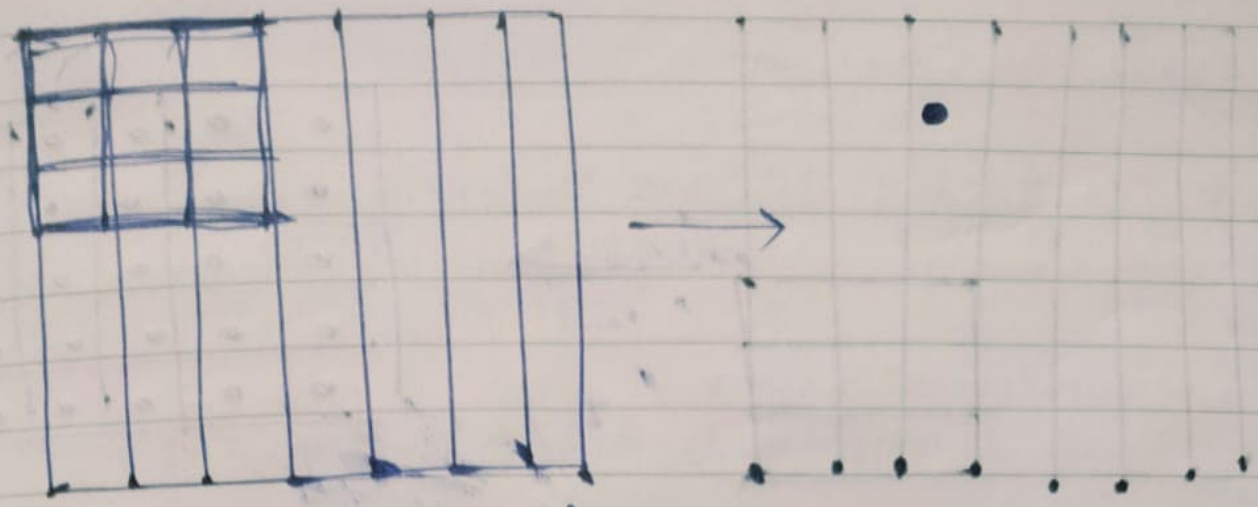
No. of filters (K)

- How to calculate (W_2, H_2).

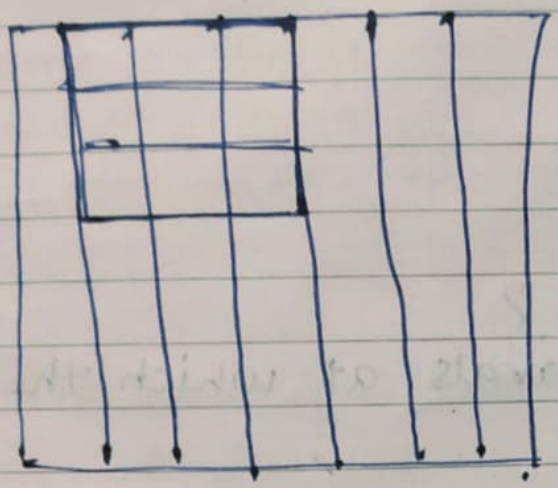
Lets take a 7×7 input and 3×3 kernels -

. . . .

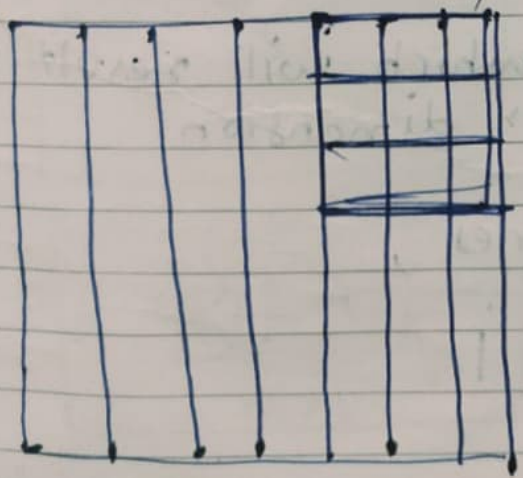
→



Move kernel →



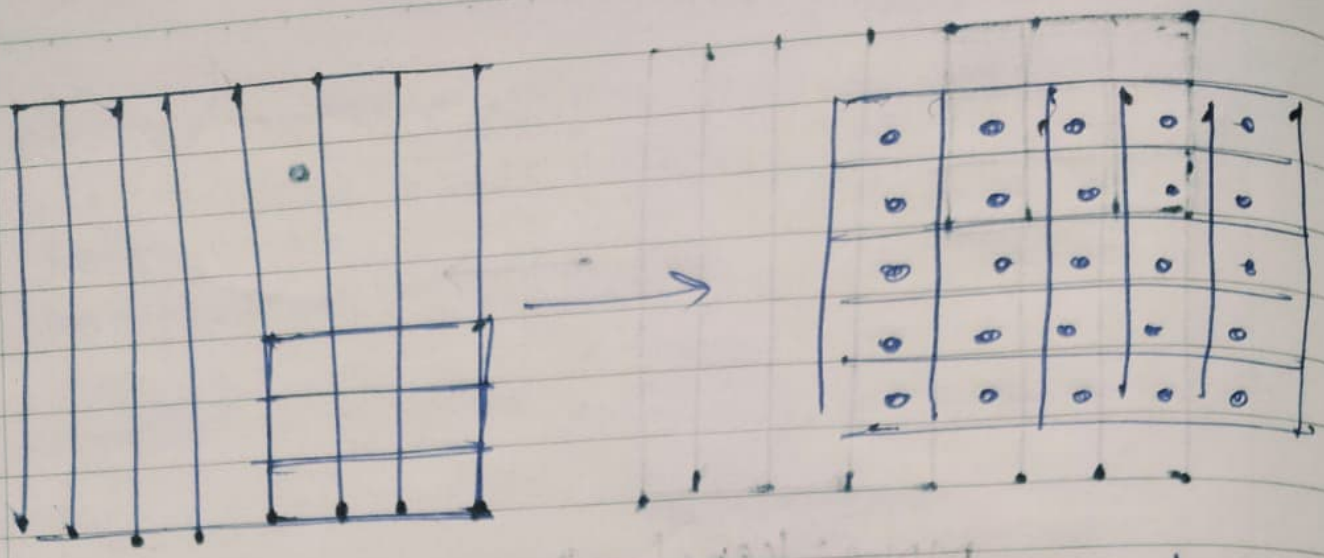
eventually



It cannot go horizontally further as it goes outside the pixel of interest.

The same is followed vertically.

final output →



The bigger the kernel, the smaller is the output.

$$W_2 = W_1 - F + 1$$
$$H_2 = H_1 - F + 1$$

- What does stride S do?
Stride defines the intervals at which the filter is applied.

So suppose, $S = 2$,
skip every 2nd pixel which will result in an output of smaller dimension.

So, the formula becomes,

$$W_2 = \frac{W_1 - F + 2P}{S} + 1$$

$$H_2 = \frac{H_1 - F + 2P}{S} + 1$$

(Replace 2P with stride interval)
i.e., padding

Output

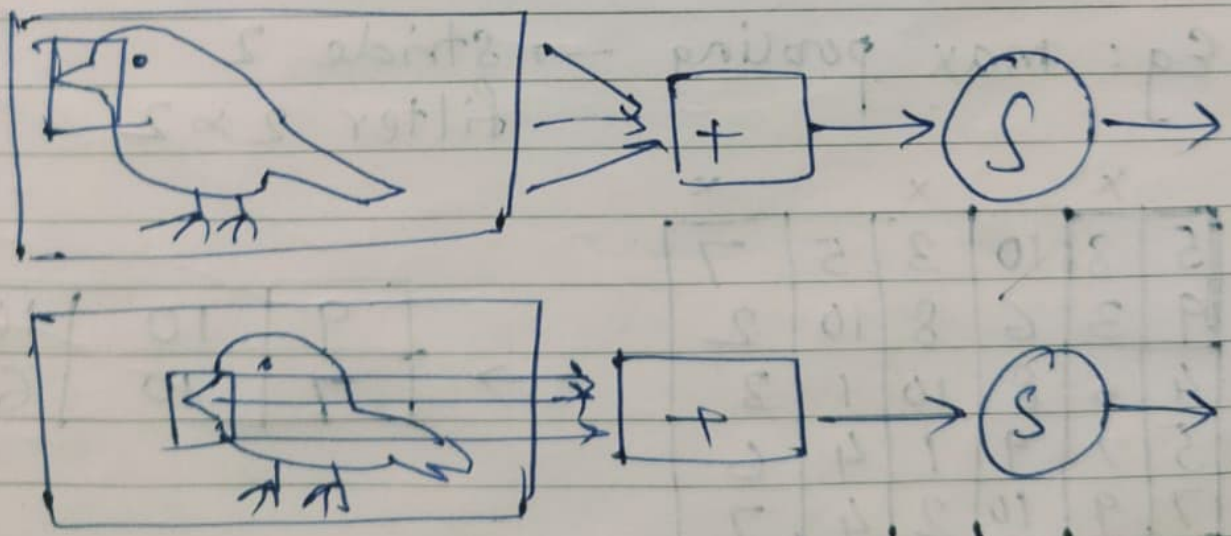
Each filter gives us one 2D output so K filters give us K such outputs

$\therefore K \times W_2 \times H_2$

$\therefore D = K$ (third dimension)

- Why do we use CNN for images?

Some patterns are much smaller than the whole image. A neuron does not have to see the whole image to discover the pattern.
We can connect to a small region with less parameters.
The same pattern appear in diff. regions.
Eg: Beak detector

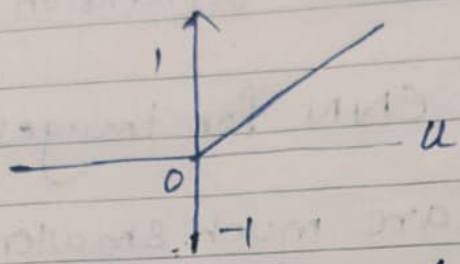


different regions but same set of parameter

CNN Architecture

1) Input Layer - accepts three dimensional input of the image with RGB color channels.

2) Convolutional Layer - ReLU function



$$f(u) = \max(0, u)$$

3) Pooling Layer

Pooling layer downsamples the volume spatially Like max pooling.

Eg: Max pooling → stride 2
filter 2×2

	5	8	10	3	5	7
×	9	3	6	8	10	2
	4	6	8	10	1	3
×	5	7	9	1	4	6
	7	9	10	2	4	7

→

9	10	10
7	10	6

if avg. pooling is applied

6	7	6
6	7	4

4) Classification layers

One or more fully connected layers to take higher-order features and produce class probabilities or scores.

fully connected to all neurons in previous layer.

The output is a 2D output [b × N] where

b → mini batch examples

N → no. of interested classes.

Numericals

Q. Let us consider a CNN having three different convolutional layers in the architecture as:

Layer 1: Filter size = 3 × 3
No. of filters = 10
Stride = 1
Padding = 0

Layer 2: Filter size = 5 × 5
No. of filters = 20
Stride = 2
Padding = 0

Layer 3: Filter size = 5 × 5
No. of filters = 40
Stride = 2
Padding = 0

If we give the input of a 3D image to the network of dimension 39 × 39. Then determine the dimension of the vector after passing through the architecture.

Sol Formula:

$$\text{Output size} = \left[\frac{W - F + 2P}{S}\right] + 1$$

Layer 1:

Input : 39 × 39 Stride = 1 Filters = 10
Filter size : 3 × 3 Padding = 0

$$\text{Output} = \left[\frac{39 - 3 + 0}{1}\right] + 1 = 37.$$

$\therefore$ output = 37 × 37 × 10

Layer 2:

Input : $37 \times 37 \times 10$ Padding = 0

Filter size : 5×5 Filters = 20

Stride = 2

$$\text{Output} = \left[\frac{37 - 5 + 0}{2}\right] + 1 = 17.$$

Output size = $17 \times 17 \times 20$

Layer 3:

Input : $17 \times 17 \times 20$ Padding = 0

Filter size : 5×5 Filters = 40

Stride = 2

$$\text{Output} = \left[\frac{17 - 5 + 0}{2}\right] + 1 = 7.$$

Output size = $7 \times 7 \times 40$.

Final flattenning = $7 \times 7 \times 40 = \underline{1960}$.

Some Popular CNN Architectures

LeNet - one of the earliest successful architectures of CNN originally used to read digits in images.

AlexNet - helped popularize CNNs in ~~no~~ computer vision.

ZF Net - introduced the visualization concept of Deconvolutional Network.

GoogleNet - codenamed 'Inception' with 22 layers

VGGNet - Showed that the depth of network was a critical factor in good performance.

ResNet - trained on very deep networks upto 1200 layers.

Drawbacks of ReLU activation function in CNN.

Dying ReLU Problem
During training, some neurons can get stuck in negative region of ReLU and never recover. These neurons become inactive - they stop learning.

(2) Non-zero Mean output
ReLU only outputs non-negative values, resulting in non-zero mean activations, which can slow down optimization and affect convergance speed.

(3) Sensitivity to learning rate
With high learning rates, ReLU units are more prone to dying or producing unstable gradients.

- **Local Receptive Fields in CNN**

A local receptive field refers to a small, localized region of the input that a neuron in the convolutional layer looks at, rather than the entire input.

Example:
Suppose you have a grayscale image of size 6×6 and you are using a 3×3 filter.

Input image (6×6)

$$\begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

First receptive field for 3×3 filter

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

- ## Weight sharing in CNN

Weight sharing means the same filter is used across all spatial locations of the input. It reduces the number of parameters significantly.

Example:
Consider your 3 x 3 filter has weights

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

When the filter is slid on a 6 x 6 input, dot product product is computed at each location.

This results in a feature map showing where the feature is found in the image.

- ## Padding and its types

Padding is the process of adding extra pixels (usually zeros) around the border of the input image before applying the convolution operation.

(1) valid Padding
No padding is added, only the valid region of the input is convolved - areas where the filter fully fits. Output size is smaller than the input.

(2) Same Padding
Padding is added so that the output size equal input size. It maintains spatial dimensions, useful in deeper networks.

- Variants of the convolution operation

(1) Standard - each output is computed by sliding a filter across the input

(2) Strided - the filter jumps with a step size (stride > 1)

(3) Dilated - introduces space between kernel elements thus increasing receptive field

(4) Transpose - reverse of convolution.

- How does CNN deal with overfitting?

(1) Regularization techniques
(2) Data augmentation (rotating, flipping, zooming, shifting and adding noise)
(3) Reducing model complexity
(4) Using more data.
(5) Transfer learning

- ## Purpose of Pooling

1. Dimensionality reduction
2. Controls overfitting
3. Retains important features